

Εφαρμογή Βαθμολογίου τριών επιπέδων (3-tier)

Τεκμηρίωση Τελικής Εργασίας — Προγραμματισμός στο Διαδίκτυο και στον Παγκόσμιο Ιστό,
Ακαδ. Έτος 2025-26

Μέλη ομάδας (Ονοματεπώνυμο)	Αριθμός Μητρώου
ΔΑΝΙΗΛ ΞΙΑΡΗ	p24218

1. Εισαγωγή

Η εργασία επεκτείνει το web project της 2ης άσκησης σε μία ολοκληρωμένη εφαρμογή τριών επιπέδων για τη διαχείριση βαθμολογίου. Υποστηρίζει τρεις κατηγορίες χρηστών — **Φοιτητές**, **Καθηγητές** και **Γραμματεία** — με σύνδεση (login) για όλους, κρυπτογραφημένους κωδικούς, διαχείριση συνόδου και αποθήκευση όλων των δεδομένων σε βάση δεδομένων.

Σύμφωνα με την υποσημείωση της 2ης άσκησης, η εφαρμογή υλοποιήθηκε ως **Spring Boot project** αντί για Dynamic Web Project. Η αντιστοίχιση των servlet και των JSP σελίδων με τα αντίστοιχα στοιχεία του Spring Boot περιγράφεται στην ενότητα 3.3.

2. Τεχνολογίες

Επίπεδο / ρόλος	Τεχνολογία
Γλώσσα	Java 21
Application server	Spring Boot 4.1.1 (Spring MVC) με ενσωματωμένο Apache Tomcat
Επίπεδο προβολής	Δυναμικές HTML σελίδες με Thymeleaf (αντί για JSP), Pico CSS για την εμφάνιση
Βάση δεδομένων	PostgreSQL 16
Σύνδεση με τη βάση	JDBC (PostgreSQL JDBC driver) μέσω του Spring JdbcTemplate
Κρυπτογράφηση κωδικών	PBKDF2-HmacSHA256 (ενσωματωμένο στη Java, χωρίς εξωτερική βιβλιοθήκη)
Εκτέλεση	Docker Compose (container για τη βάση και container για την εφαρμογή)

3. Αρχιτεκτονική

3.1 Τα τρία επίπεδα (3-tier)



Ο χρήστης στέλνει αιτήματα HTTP (GET/POST) από τον browser. Ο controller δέχεται την είσοδο, καλεί τα services, τα οποία αναζητούν ή αποθηκεύουν δεδομένα στη βάση μέσω των repositories, και το αποτέλεσμα επιστρέφεται στον χρήστη ως δυναμική HTML σελίδα.

3.2 Δομή του project

Πακέτο	Ρόλος	Κλάσεις
model	Οι βασικές κλάσεις της 1ης άσκησης και βοηθητικές κλάσεις αποτελεσμάτων	Users, Students, Professors, Secretaries, Courses, Grades, Departments, GradeSummary, SemesterGrades
controller	Δέχονται τα αιτήματα των σελίδων (ρόλος servlet)	HomeController, AuthController, StudentController, ProfessorController, SecretaryController
service	Λογική και έλεγχοι της εφαρμογής	AuthService, PasswordService, UserService, CourseService, GradeService, ProfessorService, DepartmentService
repository	Εντολές SQL προς τη βάση (JDBC)	UserRepository, StudentRepository, ProfessorRepository, CourseRepository, GradeRepository, DepartmentRepository
config	Ρυθμίσεις που εκτελούνται πριν από κάθε αίτημα	WebConfig, RoleInterceptor, NoCacheInterceptor
exception	Κεντρικός χειρισμός σφαλμάτων	GlobalExceptionHandler

3.3 Αντιστοίχιση Servlet / JSP με Spring Boot

Dynamic Web Project (servlet & JSP)	Υλοποίηση στο Spring Boot project
Servlet ανά κατηγορία χρήστη	StudentController, ProfessorController, SecretaryController — εκτελούνται μέσω του DispatcherServlet του Spring
doGet() / doPost()	Μέθοδοι με @GetMapping / @PostMapping
request.setAttribute(...) και forward σε JSP	model.addAttribute(...) και επιστροφή του ονόματος της σελίδας
JSP σελίδες	Thymeleaf templates (src/main/resources/templates)
Filter για έλεγχο πρόσβασης	RoleInterceptor (HandlerInterceptor)
HttpSession	HttpSession (το ίδιο servlet API)

Χρησιμοποιήθηκε Thymeleaf αντί για JSP επειδή είναι η προτεινόμενη τεχνολογία προβολής του Spring Boot: οι JSP σελίδες δεν υποστηρίζονται όταν η εφαρμογή εκτελείται ως εκτελέσιμο αρχείο jar (όπως στο Docker). Η λειτουργία είναι ίδια: η σελίδα δημιουργείται δυναμικά στον server από τα δεδομένα του controller.

4. Βάση δεδομένων

Πίνακας	Περιεχόμενο και σχέσεις
departments	Τμήματα. Κάθε χρήστης και κάθε μάθημα ανήκει σε ένα τμήμα (1:N).
users	Κοινά στοιχεία όλων των χρηστών: username, password_hash, salt, όνομα, επώνυμο, τμήμα, ρόλος.

students , professors , secretaries	Τα επιπλέον στοιχεία κάθε κατηγορίας (αριθμός μητρώου, κωδικός καθηγητή, αριθμός υπαλλήλου). Μοιράζονται το κλειδί του users (1:1), όπως η κληρονομικότητα των κλάσεων.
courses	Μαθήματα: κωδικός, τίτλος, ECTS, semester (1–8), τμήμα και υπεύθυνος καθηγητής (ξένο κλειδί, μπορεί να είναι κενό). Ένας καθηγητής διδάσκει 0..N μαθήματα.
enrollments	Εγγραφές φοιτητών σε μαθήματα — υλοποιεί τη σχέση πολλά-προς-πολλά.
grades	Βαθμοί (0–10). Ένας βαθμός ανά φοιτητή και μάθημα (UNIQUE).

Το πλήρες Μοντέλο Οντοτήτων-Σχέσεων βρίσκεται στο `db/er-model.drawio.xml` και η βάση (πίνακες και δοκιμαστικά δεδομένα) στο `db/studentapp_db.sql`.

Αλλαγές σε σχέση με τη 2η άσκηση

- Η στήλη `password` αντικαταστάθηκε από τις `password_hash` και `salt` (βήμα 2.1).
- Προστέθηκε η στήλη `semester` στα μαθήματα (για τη βαθμολογία ανά εξάμηνο και την προβολή μαθημάτων της γραμματείας).
- Η αρχικοποίηση της βάσης δεν διαγράφει πλέον δεδομένα: οι πίνακες δημιουργούνται μόνο αν δεν υπάρχουν (`CREATE TABLE IF NOT EXISTS`) και τα δοκιμαστικά δεδομένα προστίθενται μόνο αν λείπουν (`ON CONFLICT DO NOTHING`). Έτσι οι βαθμοί και οι αναθέσεις που καταχωρούν οι χρήστες διατηρούνται.
- Προστέθηκαν δοκιμαστικά δεδομένα: 3 τμήματα, 16 χρήστες (2 γραμματείς, 4 καθηγητές, 10 φοιτητές), 11 μαθήματα, 33 εγγραφές και 21 βαθμοί, ώστε κάθε λειτουργία να έχει δεδομένα για επίδειξη.

5. Υλοποίηση των ζητούμενων

2.1 Σύνδεση χρηστών και κρυπτογράφηση κωδικών

Όλες οι κατηγορίες χρηστών συνδέονται από την ίδια φόρμα (`POST /login`, `AuthController`). Οι κωδικοί δεν αποθηκεύονται ποτέ όπως είναι: η κλάση `PasswordService` δημιουργεί για κάθε χρήστη ένα **τυχαίο salt** 16 bytes και υπολογίζει το **hash** με τον αλγόριθμο PBKDF2-HmacSHA256 (65.536 επαναλήψεις, 256 bits). Κατά τη σύνδεση:

- το `UserRepository` βρίσκει τον χρήστη μόνο με το `username`,
- ο κωδικός που πληκτρολογήθηκε κρυπτογραφείται με το `salt` του χρήστη και συγκρίνεται με το αποθηκευμένο `hash` (σύγκριση σε σταθερό χρόνο),
- αν ταιριάζει, ο χρήστης αποθηκεύεται στο `session` — χωρίς το `hash` και το `salt`.

2.2 Κεντρικό μενού (`index.html`)

Η σελίδα `index.html` (`HomeController`) είναι η αρχική σελίδα για όλους τους χρήστες. Αν δεν έχει συνδεθεί κανείς, εμφανίζει τη φόρμα σύνδεσης· μετά τη σύνδεση εμφανίζει το μενού λειτουργιών της κατηγορίας του χρήστη. Ο `RoleInterceptor` ελέγχει πριν από κάθε σελίδα ότι ο χρήστης έχει συνδεθεί και ανήκει στη σωστή κατηγορία (`/student/**`, `/professor/**`, `/secretary/**`).

2.3 Λειτουργίες ανά κατηγορία χρήστη

Χρήστης	Λειτουργία	Controller (μέθοδος)	Σελίδα προβολής
Φοιτητής	Βαθμολογία ανά μάθημα	<code>StudentController.grades</code>	<code>student/grades.html</code>
	Βαθμολογία ανά εξάμηνο	<code>gradesBySemester</code>	<code>student/grades-by-semester.html</code>
	Συνολική βαθμολογία	<code>summary</code>	<code>student/summary.html</code>

Καθηγητής	Λίστα βαθμολογίας ανά μάθημα (βαθμολογημένα)	<code>ProfessorController.grades</code>	<code>professor/grades.html</code>
	Καταχώρηση βαθμολογίας (μη βαθμολογημένα)	<code>enterGradesForm / enterGrades</code>	<code>professor/enter-grades.html</code>
Γραμματεία	Προβολή μαθημάτων (τίτλος, εξάμηνο, υπεύθυνος καθηγητής)	<code>SecretaryController.courses , coursesWithProfessors</code>	<code>secretary/courses.html , secretary/courses-professors.html</code>
	Ανάθεση μαθήματος σε καθηγητή	<code>assignForm / assign</code>	<code>secretary/assign.html</code>
	Δημιουργία φοιτητών και καθηγητών (1η άσκηση, 10.1.1)	<code>createUserForm , createStudent , createProfessor</code>	<code>secretary/create-user.html</code>
	Δημιουργία μαθημάτων (1η άσκηση, 10.1.1)	<code>createCourseForm / createCourse</code>	<code>secretary/create-course.html</code>
	Λίστες φοιτητών προς βαθμολόγηση (1η άσκηση, 10.1.3)	<code>enrollments / enroll</code>	<code>secretary/enrollments.html</code>

- **Συνολική βαθμολογία φοιτητή** (`GradeSummary`): μέσος όρος των μαθημάτων που πέρασε (βαθμός ≥ 5), μέσος όρος όλων των μαθημάτων που εξετάστηκε, ECTS, πλήθος επιτυχόντων και αποτυχόντων.
- **Καταχώρηση βαθμών** (`GradeService.saveGrades`): ελέγχεται ότι το μάθημα έχει ανατεθεί στον συγκεκριμένο καθηγητή, ότι κάθε φοιτητής είναι εγγεγραμμένος και δεν έχει ήδη βαθμό, και ότι ο βαθμός είναι από 0 έως 10. Αν κάποιος έλεγχος αποτύχει, δεν αποθηκεύεται κανένας βαθμός (transaction).
- Ένας καθηγητής δεν μπορεί να δει ή να βαθμολογήσει μαθήματα άλλου καθηγητή, ακόμα κι αν αλλάξει το `courseId` στη διεύθυνση.
- **Επιπλέον λειτουργίες γραμματείας** (βήματα 10.1.1 και 10.1.3 της 1ης άσκησης): δημιουργία φοιτητών και καθηγητών (`UserService` — ο αρχικός κωδικός αποθηκεύεται ως `hash + salt`), δημιουργία μαθημάτων και εγγραφή φοιτητών σε μαθήματα. Οι εγγραφές αποτελούν τις λίστες φοιτητών που βαθμολογεί ο καθηγητής. Ελέγχεται ότι το όνομα χρήστη, ο αριθμός μητρώου, ο κωδικός καθηγητή και ο κωδικός μαθήματος δεν υπάρχουν ήδη.

2.4 Διαχείριση συνόδου (session management)

- Μετά τη σύνδεση ο χρήστης αποθηκεύεται στο `HttpSession` και δημιουργείται **νέο session id** (`changeSessionId`), για προστασία από session fixation.
- Το session αναγνωρίζεται μόνο από cookie και λήγει μετά από 30 λεπτά αδράνειας.
- Κατά την **αποσύνδεση** το session διαγράφεται (`session.invalidate()`) και στέλνεται η επικεφαλίδα `Clear-Site-Data: "cache"`, ώστε ο browser να διαγράψει την cache της εφαρμογής.
- Ο `NoCacheInterceptor` προσθέτει σε κάθε σελίδα τις επικεφαλίδες `Cache-Control: no-cache, no-store, must-revalidate, Pragma` και `Expires`. Έτσι, μετά το logout, το κουμπί «Πίσω» του browser δεν εμφανίζει σελίδες του προηγούμενου χρήστη.

3.1 & 4.1 Επίπεδο δεδομένων και επίπεδο προβολής

Όλα τα δεδομένα αποθηκεύονται στη βάση PostgreSQL (ενότητα 4). Για κάθε λειτουργία υπάρχει μία σελίδα Thymeleaf που λαμβάνει τα αποτελέσματα από τον αντίστοιχο controller και δημιουργεί δυναμικά την προβολή. Το κοινό μενού βρίσκεται στο `fragments/layout.html` και τα σφάλματα εμφανίζονται στη σελίδα `error.html` (`GlobalExceptionHandler`).

6. Εκτέλεση της εφαρμογής

Με **Docker** (από τον φάκελο του project):

```
docker compose up --build
```

και άνοιγμα του `http://localhost:8080`.

Στην πρώτη εκκίνηση οι πίνακες και τα δοκιμαστικά δεδομένα δημιουργούνται αυτόματα. Για επαναφορά της βάσης στην αρχική κατάσταση: `docker compose down -v` ή φόρτωση του `db/studentapp_db.sql`.

Από το IDE: εκτέλεση της `StudentappApplication` με βάση PostgreSQL στη θύρα 5433 (ρυθμίσεις στο `application.properties`).

Δοκιμαστικοί λογαριασμοί

Κατηγορία	Username	Κωδικός
Γραμματεία	gramm1	gramm123
Καθηγητής	kkosta, mpapa, gdimou, avasil	prof123
Φοιτητής	sofia.n, giorgos.p κ.ά.	stud123

7. Παραδοχές

- Βαθμολογημένο μάθημα:** η εκφώνηση δεν ορίζει πότε ένα μάθημα θεωρείται βαθμολογημένο. Θεωρήσαμε ότι ένα μάθημα εμφανίζεται στη «λίστα βαθμολογίας» όταν έχει τουλάχιστον έναν βαθμό, και στην «καταχώρηση βαθμολογίας» όσο υπάρχουν εγγεγραμμένοι φοιτητές χωρίς βαθμό. Έτσι ένα μάθημα μπορεί να βαθμολογηθεί σταδιακά.
- Επιτυχία σε μάθημα:** βαθμός μεγαλύτερος ή ίσος του 5.
- Εξάμηνια:** από 1 έως 8.

Παράρτημα: Συχνές ερωτήσεις για την εφαρμογή

1. Κρυπτογράφηση κωδικών (hash + salt)

Χρησιμοποιείται ο αλγόριθμος **PBKDF2 με HmacSHA256**, ενσωματωμένος στη Java (`javax.crypto.SecretKeyFactory`), χωρίς εξωτερική βιβλιοθήκη. Για κάθε χρήστη παράγεται τυχαίο **salt** 16 bytes με `SecureRandom` και το hash υπολογίζεται με 65.536 επαναλήψεις (μήκος 256 bits). Στη βάση αποθηκεύονται μόνο το hash και το salt, ποτέ ο πραγματικός κωδικός. Υλοποίηση: `PasswordService` (`generateSalt`, `hash`, `matches`).

- **Γιατί salt;** Όστε δύο χρήστες με τον ίδιο κωδικό να έχουν διαφορετικό hash και να μην λειτουργούν έτοιμοι πίνακες hash (rainbow tables).
- **Γιατί πολλές επαναλήψεις;** Κάνουν αργό τον υπολογισμό, άρα ασύμφορη την προσπάθεια εύρεσης του κωδικού με δοκιμές.
- **Πώς γίνεται ο έλεγχος στο login;** Ο χρήστης αναζητείται **μόνο με το username**, ο κωδικός που πληκτρολογήθηκε κρυπτογραφείται με το salt του χρήστη και τα δύο hash συγκρίνονται με `MessageDigest.isEqual` (σύγκριση σταθερού χρόνου).

2. Prepared statements και SQL injection

Όλα τα ερωτήματα γράφονται με `?` και οι τιμές στέλνονται ξεχωριστά μέσω `JdbcTemplate`, π.χ. `WHERE u.username = ?` στο `UserRepository`. Η βάση παραλαμβάνει πρώτα το ερώτημα και μετά τις τιμές, οπότε μια τιμή δεν μπορεί ποτέ να γίνει εντολή SQL.

- **Πού θα υπήρχε κίνδυνος;** Αν το ερώτημα φτιαχνόταν με ένωση κειμένου, π.χ. `"... WHERE username = '" + username + "'"`.
- **Παράδειγμα επίθεσης:** με είσοδο `' OR '1'='1` η συνθήκη θα γινόταν πάντα αληθής και θα ήταν δυνατή η σύνδεση χωρίς κωδικό. Με prepared statement, η βάση ψάχνει χρήστη με αυτό ακριβώς το όνομα και δεν βρίσκει κανέναν.
- **Προστασία από XSS:** οι σελίδες τυπώνουν τιμές με `th:text`, που μετατρέπει τους χαρακτήρες HTML σε ακίνδυνο κείμενο.

3. Σύνδεση με τη βάση δεδομένων

Η επικοινωνία γίνεται με **JDBC**, με τον PostgreSQL driver που δηλώνεται στο `pom.xml`. Τα στοιχεία σύνδεσης βρίσκονται στο `application.properties` και το Spring Boot δημιουργεί από αυτά το `JdbcTemplate`, το οποίο δίνεται σε κάθε repository μέσω του constructor.

- **Τι κάνει το JdbcTemplate;** Ανοίγει τη σύνδεση, δημιουργεί το `PreparedStatement`, εκτελεί το ερώτημα, μετατρέπει τις γραμμές σε αντικείμενα με έναν `RowMapper` και κλείνει τα πάντα, ακόμα και σε περίπτωση σφάλματος.
- **Connection pool:** χρησιμοποιείται το HikariCP (προεπιλογή του Spring Boot), ώστε οι συνδέσεις να επαναχρησιμοποιούνται αντί να ανοίγουν από την αρχή σε κάθε ερώτημα.
- **Πού τρέχει η βάση;** Σε container PostgreSQL 16 (θύρα 5432 εσωτερικά, 5433 στον υπολογιστή). Οι πίνακες και τα δοκιμαστικά δεδομένα δημιουργούνται στην εκκίνηση από τα `schema.sql` και `data.sql`.

4. Διαχείριση συνόδου (session)

Το HTTP δεν έχει μνήμη, οπότε ο server χρησιμοποιεί session: κατά το login ο χρήστης αποθηκεύεται στο `HttpSession` και ο browser κρατάει μόνο το αναγνωριστικό (cookie `JSESSIONID`). Τα δεδομένα παραμένουν στον server, άρα ο ρόλος δεν μπορεί να αλλοιωθεί από τον χρήστη.

- **Έλεγχος πρόσβασης:** ο `RoleInterceptor` τρέχει πριν από κάθε προστατευμένη σελίδα και ελέγχει το session. Αν ο χρήστης δεν έχει συνδεθεί ή ανήκει σε άλλη κατηγορία, η σελίδα δεν εκτελείται ποτέ.

- **Ασφάλεια συνόδου:** μετά το login δημιουργείται νέο session id (`changeSessionId`) για προστασία από session fixation, το session αναγνωρίζεται μόνο από cookie και λήγει μετά από 30 λεπτά αδράνειας.
- **Αποσύνδεση:** `session.invalidate()` και επικεφαλίδα `Clear-Site-Data: "cache"`, ενώ ο `NoCacheInterceptor` εμποδίζει τον browser να αποθηκεύσει σελίδες, ώστε το κουμπί «Πίσω» να μην δείχνει δεδομένα του προηγούμενου χρήστη.

5. Παράδειγμα πλήρους διαδρομής: καταχώρηση βαθμού

1. Ο καθηγητής υποβάλλει τη φόρμα: `POST /professor/enter-grades`.
2. `NoCacheInterceptor` και `RoleInterceptor` ελέγχουν τη σύνοδο και τον ρόλο.
3. `ProfessorController.enterGrades` διαβάζει τα πεδία της φόρμας.
4. `GradeService.saveGrades` ελέγχει ότι το μάθημα ανήκει στον καθηγητή, ότι κάθε φοιτητής είναι εγγεγραμμένος και αβαθμολόγητος και ότι ο βαθμός είναι 0–10. Η μέθοδος είναι `@Transactional`: ή αποθηκεύονται όλοι οι βαθμοί ή κανένας.
5. `GradeRepository.insert` εκτελεί το `INSERT` με prepared statement.
6. Ο controller επιστρέφει ξανά τη σελίδα με μήνυμα επιτυχίας ή σφάλματος, την οποία παράγει το Thymeleaf.